

Chapter 10

Configuring and Securing SSH

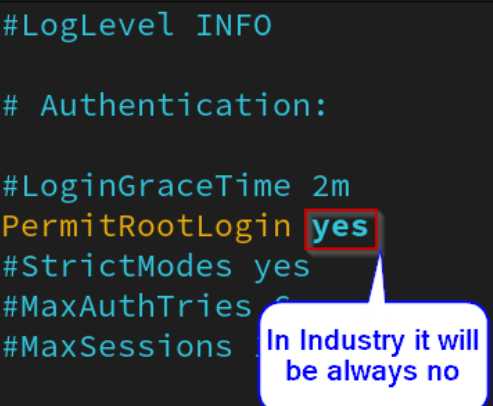
ssh command to create a **secure connection to a remote system, authenticate as a specific user, and get an interactive shell session** on the remote system as that user.

You may also use the **ssh** command to run an **individual command** on the remote system without running an interactive shell.

-
- SSH config file is located at **/etc/ssh/sshd_config**.
- The **.ssh** directory in Linux is a hidden folder located in a user's home directory (e.g., **/home/username/.ssh**) that stores **SSH (Secure Shell) configuration files and keys used for secure remote login and communication**.
- It holds SSH keys like the private key (**id_rsa**) and the public key (**id_rsa.pub**) used for key-based authentication.
- It contains the **authorized_keys** file, which lists public keys allowed to access the account remotely.
- It stores known host keys in the **known_hosts** file, which helps verify the identity of remote servers to prevent man-in-the-middle attacks.

How to access SSH :-

```
[root@Server ~]# vi /etc/ssh/sshd_config
```



```
#LogLevel INFO
# Authentication:
#LoginGraceTime 2m
PermitRootLogin yes
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10
```

```
[root@Server ~]# systemctl restart sshd.service
```

This is How we can access our SSH Server using the following command

```
[root@Client ~]# ssh root@192.168.158.128
```

```
[root@Client ~]# ssh root@192.168.158.128
root@192.168.158.128's password:
Activate the web console with: systemctl enable --now cockpit.socket

Last failed login: Mon Sep  8 17:47:40 IST 2025 from 192.168.158.131 on ssh:notty
There were 2 failed login attempts since the last successful login.
Last login: Mon Sep  8 14:58:26 2025
[root@Server ~]#
```

To Close the SSH Connection #logout command is used

```
[root@Server ~]# logout
Connection to 192.168.158.128 closed.
[root@Client ~]#
```

Configuring SSH Key-based Authentication

You can configure an SSH server to allow you to **authenticate without a password by using keybased authentication. This is based on a **private-public key scheme.****

Generating SSH Key

✓ From Client you have to generate keys

```
[root@Client ~]# ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Created directory '/root/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:zgyPmShrtXGpe3hgNzdy8vix7FqMaD6D7oKby9cy7G0 root@Client
The key's randomart image is:
+----[RSA 3072]-----+
|
|
|
|
| ..S
| =oB%+
|. o+=O=OO.
|+.oXBE+o.o
|+O*,*B.+.=
+----[SHA256]-----+
[root@Client ~]#
```

```
[root@Client ~]# ssh-copy-id root@192.168.158.128
The authenticity of host '192.168.158.128 (192.168.158.128)' can't be established.
ED25519 key fingerprint is SHA256:RdG9r7cpgDIFcm0zHKJgy9CQZ+vwToLAgk3vL7TIT+w.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new ke
ys
root@192.168.158.128's password:
Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'root@192.168.158.128'"
and check to make sure that only the key(s) you wanted were added.

[root@Client ~]#
```

Once SSH Key is Copy successfully you can now login to your Server without prompting any password

```
[root@Client ~]# ssh root@192.168.158.128
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Mon Sep  8 17:59:21 2025 from 192.168.158.131
[root@Server ~]#
```